

作业 13

实验环境：

- 开发环境：Python 编程语言
- 运行环境：Windows11
- 工具：Visual Studio Code
- 库：math , matplotlib.pyplot , numpy

实验内容：

用Metropolis – Hasting抽样方法计算积分： $I = \int_0^{\infty} (x - \alpha\beta)^2 f(x) dx = \alpha\beta^2$

$$f(x) = \frac{1}{\beta\Gamma(\alpha)} \left(\frac{x}{\beta}\right)^{\alpha-1} \exp(-x/\beta)$$

设积分的权重函数为： $p(x) = f(x)$ 和 $p(x) = (x - \alpha\beta)^2 f(x)$ 给定参数 $\alpha\beta$ ，
并用不同的 γ 值，分别计算积分，讨论计算精度和效率

理论计算：

$$\text{细致平衡条件: } \frac{p_i}{p_j} = \frac{T_{ij}A_{ij}}{T_{ji}A_{ji}}$$

$$A_{ij} = \min\{1, \frac{p_j T_{ji}}{p_i T_{ij}}\}$$

此处 p 为伽马分布 $p(x) = \frac{\lambda^a x^{a-1} e^{-\lambda x}}{\Gamma(a)}$ ，表示发生 a 次所用时间的概率

$$\text{均值为 } \frac{a}{\lambda}, \text{ 方差为 } \frac{a}{\lambda^2}$$

$T(x \rightarrow x')$ 可取任意条件概率分布，按照题意，取：

1. 均值为 x 的指数分布
2. 均值为 x 的高斯分布
3. 均值待定的指数分布

分别对应代码里的三部分

代码流程：

```
1. import numpy as np
2. import math
3. import matplotlib.pyplot as plt
4.
5.
6. aaaaaaaaa = 16807
7. m = 2147483647
8.
9. def azmodm(z):
10.     global m, aaaaaaaaa
11.     q = int(m / aaaaaaaaa)
12.     r = m % aaaaaaaaa
```

```

13.     tmp = aaaaaaaaa * (z % q) - r * int(z / q)
14.     if tmp >= 0:
15.         return tmp
16.     else:
17.         return tmp + m
18.
19. bbbssbbb = 0
20. aaassaaa = 0
21. cccssccc = 0
22.
23. def random01():
24.     global bbbssbbb, aaassaaa, m, cccssccc
25.     z1 = 666955
26.     z2 = 816815
27.     z3 = 116560
28.     z4 = 4813144
29.     if aaassaaa == 0 and cccssccc == 0:
30.         bbbssbbb = z1
31.     if aaassaaa == m - 1000000 and cccssccc == 0:
32.         bbbssbbb = z2
33.         aaassaaa = 1
34.         cccssccc = 1
35.     if aaassaaa == m - 1000000 and cccssccc == 1:
36.         bbbssbbb = z3
37.         aaassaaa = 1
38.         cccssccc = 2
39.     if aaassaaa == m - 1000000 and cccssccc == 2:
40.         bbbssbbb = z4
41.         aaassaaa = 1
42.         cccssccc = 3
43.     bbbssbbb = azmodm(bbbssbbb)
44.     aaassaaa += 1
45.     return bbbssbbb / m
46.
47. def FF(x,lam):##指数法分布
48.     return max(lam*np.exp(-lam*x),0)
49. def Gs(x,xbar):##高斯分布
50.     return (np.exp(-(x-xbar)**2/2))/np.sqrt(2*np.pi)
51.
52.
53. a=int(13) ##根据伽马函数的意义，此处设 a 为正整数
54. lam=0.5
55. E=a/lam
56. F=a/lam**2

```

```

57.
58. def PP(x): ##权重函数 1
59.     # print(x)
60.     if x>=0:
61.         return (lam**a)*(x**(a-1))*np.exp(-lam*x)/math.factorial(a)
62.     else:
63.         return 0
64. def pp(x): ##画图用
65.     return (lam**a)*(x**(a-1))*np.exp(-lam*x)/math.factorial(a)
66.
67. # def PP(x): ##权重函数 2
68. #     # print(x)
69. #     if x>=0:
70. #         return (lam**a)*(x**(a-1))*np.exp(-lam*x)/math.factorial(a)*(x-E)**2
71. #     else:
72. #         return 0
73. # def pp(x): ##画图用
74. #     return (lam**a)*(x**(a-1))*np.exp(-lam*x)/math.factorial(a)*(x-E)**2
75. i=0
76. j=0
77. x=0
78. c=1 ##初始的 c 值
79. N=100000
80. lam2=1/E
81.
82. for j in range(3):
83.     p=[]
84.     p.append(E) ##设置初始值
85.     i=0
86.     while(not(len(p)>=N)):
87.         if j==0:
88.             x=np.log(random01())/(-1/p[i])
89.             c=min(PP(x)*FF(p[i],1/x)/(PP(p[i])*FF(x,1/p[i])),1) ##转移函数 1
90.
91.         if j==1:
92.             x=p[i]+(np.sqrt(-2*np.log(random01()))*np.cos(2*np.pi*random01()))
93.             c=min(PP(x)*Gs(p[i],x)/(PP(p[i])*Gs(x,p[i])),1) ##转移函数 2
94.
95.         if j==2:
96.             x=np.log(random01())/(-lam2)
97.             c=min(PP(x)*FF(p[i],lam2)/(PP(p[i])*FF(x,lam2)),1) ##转移函数 3
98.             R=random01()
99.             if c>R:
100.                 p.append(x)

```

```

101.         else:
102.             p.append(p[i])
103.             i+=1
104.
105.         h=sum((np.array(p)-E)**2)/len(p)
106.         d=sum((np.array(p)))/len(p)
107.         print((h),F,abs(F-h),d,E,abs(d-E))
108.
109.         # h=sum((np.array(p)))/len(p)
110.         # d=sum((np.array(p)))/len(p)
111.         # print((h),E,abs(F-h))
112.
113.
114.         x=np.array(p)
115.         plt.figure() #初始化一张图
116.         width = 100 #区间数
117.         n, bins, patches = plt.hist(x,bins = width,range=(min(p),max(p)),density=True)
118.         # print(x)
119.         # print(n)
120.         # print(bins)
121.         # print(patches)
122.         # plt.grid(alpha=0.5,linestyle='-.') #网格线，更好看
123.         plt.xlabel('X')
124.         plt.ylabel('f(X)')
125.         # plt.title(r'频率分布直方图')
126.         plt.xticks(np.arange(int(min(p)),int(max(p)),6))
127.         z=np.linspace((min(p)),(max(p)))
128.         z=np.array(z)
129.         zzz=pp(z)/max(pp(z))*max(n) ##此处将 PP(x)放大，max 为其最值，便于观察
130.         plt.scatter(z,zzz,color='orange')
131.     plt.show()

```

代码中定义了两个 PP(x)，其中一组被注释掉，分别代表两种权重函数；代码中定义了转移函数 1, 2, 3，分别代表三种不同的接受函数

结果讨论：

经过尝试，最终发现，设置使得指数函数的均值与伽马分布的均值相近时（代码中使相等了，但最佳时并不是完全相等，但结果相差不大，所以使相等了）结果较好。

另外，由于转移函数 1, 2 不受参数 lam2 的影响，而受初始值 p【0】的影响较大，经过尝试发现，令 p【0】与伽马分布的均值相近时（理论上应该放在伽马分布概率最大处较好，这并不等于均值，但两者相差不大，所以不妨放在均值处，此外，初始值对高斯分布影响较大，对均值为 x 的指数分布影响较小）结果较好。

打印结果：

```

51.20424303839014 52.0 0.795756961609861 25.994252530661715 26.0 0.005747469338285072
57.3414235818953 52.0 5.341423581895299 26.398335891491254 26.0 0.3983358914912536

```

52.33939758638373 52.0 0.33939758638373263 25.999928944498066 26.0 7.105550193386989e-05

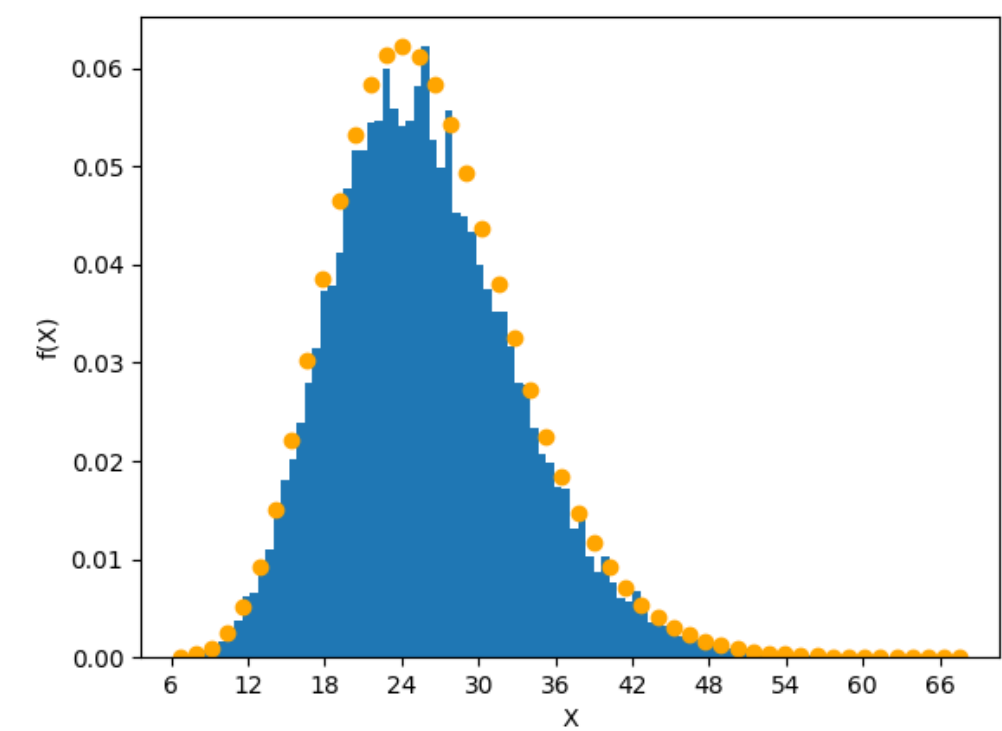


图 1 均值为 x 的指数分布

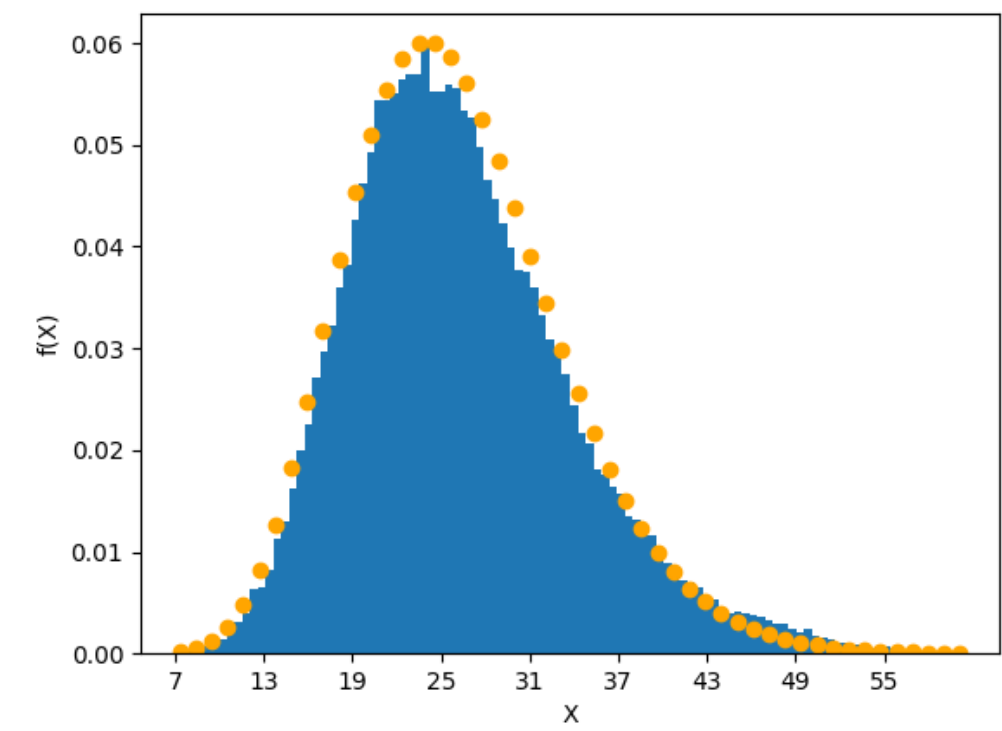


图 2 均值为 x 的高斯分布

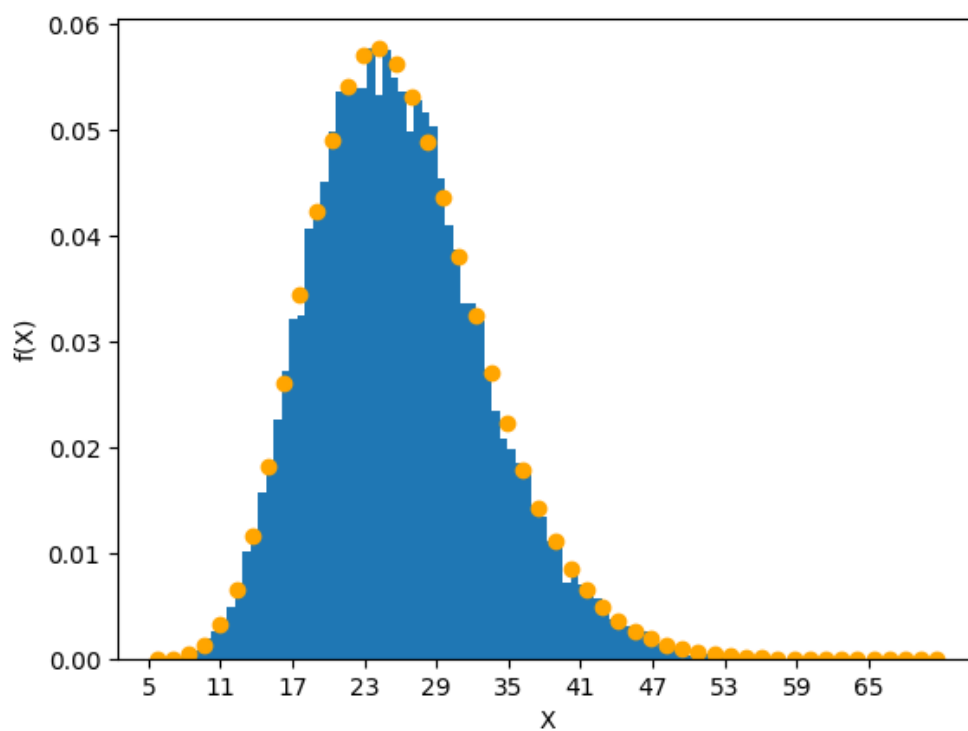


图 3 均值待定的指数分布

由结果可知：

均值为 x 的指数分布、均值为 x 的高斯分布、均值待定的指数分布

三种中，第一、三种转移函数结果较好，高斯分布尽管抽出来的结果较光滑，但方差与精确值相差较大。